Pip Architecture — F6: pgvector semantic recall

FOLIOS · FOLIOSHQ.COM

Source: `docs/audit-2026-06-17.md` → X6 Pip Architecture Review ("FORM" sequence) and GAME PLAN Phase 2 #5 (portable-brain depth — "semantic search over all notes/summaries"). F1 (shared `buildAccountContext()`) shipped June 19; F2/F3 (persist structured context + event-driven recompute) shipped June 19. F6 is the last memory-architecture step before F5 (agent loop).

The goal: Pip can pull relevant past context by **meaning**, not just recency — *"what did we decide about the invoice-feed integration six months ago?"* across all accounts — instead of only ever seeing the last N meetings by date.

Work on the branch (`claude/f6-pgvector-recall-jh2yjc`); do **NOT** deploy. Chris fast-forwards `main` after he tests (the F1/F3 playbook). No destructive DB changes.

WHERE IT PLUGS IN (the F1 dividend)

F1 made `src/lib/accountContext.js#buildAccountContext()` the **single** per-account context builder that chat, summarize, and operator all read from. F6 adds **one new section** to that builder — `recall` — so any surface that supplies recall hits renders them identically, by construction (the parity rule). The fetch (embed the query → vector search) is async and surface-specific, so the *caller* attaches `account.recallHits`; the *builder* only renders them. v1 wires the fetch into **chat** (the natural "what did we decide about X" surface); summarize/operator can attach hits later with **zero render changes**.

```
chat (api/pip.js)
  ├─ embed(lastUserMessage)                      ← 1 cheap OpenAI
embed call
  ├─ match_folio_embeddings(qvec, account, k)    ← per focused account
→ a.recallHits
  └─ match_folio_embeddings(qvec, null, k)       ← global lane (list
mode) → curated.globalRecall
        ↓
  buildAccountContext(a,{surface:"chat"})  →
recallSection(a.recallHits)   ← THE shared render
```

WHAT GETS EMBEDDED (and the DATA LINE)

The recall corpus is **the user's own notebook + Pip's already-governed distillations** — four source types, all loaded from tables the app already owns:

source_type	source	author	data-line nc
`meeting_notes`	`folio_meetings.notes`	user	verbatim – notebook. is (never e
`meeting_summary`	`folio_meetings.pip_summary`	Pip	Pip-authore **generaliz** generation Data Line F revenue/vo Embedding guarantee.
`project_note`	`folio_meetings.project_notes[projectId]`	user	verbatim p notes type
`account_update`	`folio_account_updates.title + .description`	user	verbatim.

Pip Data Line Rule compliance (compliance-critical):

- Raw **user** text (`notes` , `project_notes` , `account_update`) is the user's own words — embedding it stores nothing Pip authored, so the "never retain" half doesn't apply (it's the same as the verbatim text already sitting in `folio_meetings.notes`).
- The one **Pip-authored** source (`pip_summary`) is already required to generalize any quantitative business data at generation time (the summarize prompts enforce it). F6 does not introduce any *new* Pip-authored retention — it embeds text that already exists under the rule. `docs/data-handling.md` is updated to state exactly what's embedded and the RLS scope **before** this ships.
- F6 **never** sends embedded text to any Pip surface that the user couldn't already see for that account — recall is RLS-scoped to the user and (by default) account-scoped.

Explicitly **not** embedded in v1: `gauge_projects.notes` / `status_updates` (already in the recency context + lower recall value), contacts, tasks, glossary (structured, not prose).

SCHEMA

`vector` extension is available on the project (`v0.8.0` , supports hnsw up to 2000 dims). Embedding model: **OpenAI** **`text-embedding-3-small`** → `vector(1536)` (see "Embedding provider" below for the justification + the env-swappable alternative).

```sql
create extension if not exists vector;

create table if not exists folio_embeddings (
  id                  uuid primary key default gen_random_uuid(),
  user_id             uuid not null references auth.users(id) on
delete cascade,
  source_type         text not null,        -- meeting_notes |
meeting_summary | project_note | account_update
  source_id           text not null,        -- meeting id,
meetingId:projectId, or update id
  account_id          uuid references folio_accounts(id) on delete
cascade,  -- nullable, but always set in v1
  chunk_index         int  not null default 0,
  content             text not null,        -- the chunk text
actually embedded
  content_fingerprint text not null,        -- hash(content) of
the WHOLE source row — skip re-embed when unchanged
  embedding           vector(1536) not null,
  created_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now(),
  unique (user_id, source_type, source_id, chunk_index)
);

alter table folio_embeddings enable row level security;
create policy "embeddings_owner_select" on folio_embeddings
  for select using ((select auth.uid()) = user_id);
create policy "embeddings_owner_write"  on folio_embeddings
  for all using ((select auth.uid()) = user_id) with check
((select auth.uid()) = user_id);

create index if not exists folio_embeddings_user_account on
folio_embeddings(user_id, account_id);
create index if not exists folio_embeddings_source       on
folio_embeddings(user_id, source_type, source_id);
create index if not exists folio_embeddings_hnsw
  on folio_embeddings using hnsw (embedding vector_cosine_ops);
```

Why hnsw, not ivfflat: the dataset is tiny (single user, tens of accounts) and grows slowly. hnsw needs no `lists` tuning and no training rows to get good recall; ivfflat is worse on small/empty tables. Trade-off (slower index build at scale) is irrelevant here.

Recall RPC — one function, `SECURITY INVOKER` (so RLS applies) **and** an explicit `user_id = auth.uid()` predicate (defense in depth — a mis-scoped vector query is a silent cross-user leak, the exact Sanity-Pass failure class):

```
create or replace function match_folio_embeddings(
  query_embedding vector(1536),
  match_account   uuid default null,
  match_count     int  default 5
) returns table (
  source_type text, source_id text, account_id uuid, content text,
similarity float
)
language sql stable security invoker set search_path = public as
$$
  select e.source_type, e.source_id, e.account_id, e.content,
         1 - (e.embedding <=> query_embedding) as similarity
  from folio_embeddings e
  where e.user_id = (select auth.uid())                          --
belt (RLS is the suspenders)
    and (match_account is null or e.account_id = match_account)
  order by e.embedding <=> query_embedding
  limit greatest(1, least(match_count, 20));
$$;
grant execute on function match_folio_embeddings(vector, uuid,
int) to authenticated;
```

The RPC is always called via the **user-JWT** client (Supabase Server-Client Auth Rule), so `auth.uid()` resolves and both the RLS policy and the explicit predicate are active.

PIPELINE — embed once, never re-embed unchanged

Writer: `api/embed-sync.js` (new handler)

`POST { accountIds?: string[] }` (JWT auth; falls back to all the caller's active accounts, capped). For the account set:

1. Load `folio_meetings` (id, account_id, notes, pip_summary, project_notes) + the project title map, and `folio_account_updates` (id, account_id, title, description) — via the user-JWT client (RLS).
2. Build candidate **source rows** → `{ source_type, source_id, account_id, content }` (one per meeting-notes / meeting-summary / project-note / update). Skip empties + trivially short content (< 24 chars).
3. `content_fingerprint = fnv1a(content)`. Load existing `folio_embeddings` fingerprints for these accounts; **skip** any source whose stored fingerprint matches (the embed-once guarantee — reuses the F3 fingerprint idea applied per source row).
4. Chunk long content (~1500 chars on paragraph boundaries, cap 8 chunks/source); most sources are 1 chunk.
5. Embed only the changed/new chunks — **one batched OpenAI call** per ≤96 chunks.
6. For each changed source: delete its old chunks, insert the new ones (idempotent on the unique key). Log embedding spend via `logPipUsage` (model added to `_pipUsage.js` cost table). Return `{ embedded, skipped }`.

Trigger: `src/hooks/useEmbeddingSync.js` (new hook, called once in App.jsx, above the `authLoading` return → Hook Order Rule)

- Runs **once per day per user** (localStorage throttle), fire-and-forget, posting the active account ids. First run = the one-time **backfill**; every later run is a

cheap catch-up that the per-source fingerprint gate makes nearly free (it just SELECTs fingerprints and skips).

- No-ops cleanly when `userId` is null or `OPENAI_API_KEY` isn't configured.
- Recall targets **old** content; same-day freshness is not required (recent meetings are already in the recency context), so a daily sweep is sufficient. An after-summarize trigger for instant recall of brand-new notes is a noted fast-follow, not v1.

Reader: recall in `api/pip.js` (chat/brief only)

- Gate: `mode ∈ {chat, brief}`, `lastUserMessage.length ≥ 12`, `OPENAI_API_KEY` present. Otherwise recall is skipped entirely (no cost, no behavior change).
- Embed `lastUserMessage` once. Then:
  - **focused mode** → for each focused account, `match_folio_embeddings(qvec, account.id, 4)` → `account.recallHits`. (Recall already-loaded source rows are de-emphasized: hits whose text is already verbatim in the rendered recent-meeting block add little but cost little; K is small.)
  - **list mode** (nothing focused — the "across all accounts" case) → `match_folio_embeddings(qvec, null, 6)` → `curated.globalRecall`, rendered as a portfolio-level "RELEVANT PAST CONTEXT" block.

- Token budget: each hit truncated to ~280 chars; K ≤ 6. Worst case ≈ 1.7k extra context chars per chat turn — cheap and bounded.

Render: `accountContext.js#recallSection` + `pipContext.js` global block

- New `recall` section in `SECTION_ORDER`, `includeRecall` default **true** for `chat` / `brief`, **false** for `summarize` / `operator` (cost control;

they don't fetch hits in v1 anyway).

- Reads `a.recallHits = [{ content, source_type, source_id, date?, similarity }]`; renders a compact "RELEVANT PAST NOTES (semantic recall — older context surfaced by meaning)" block, labeled by source type + date, so the model can tell recalled context from current context.
- The global lane renders in `renderContextProse` (portfolio-level), deduped against any focused-account hits by `source_id`.

EMBEDDING PROVIDER (and cost)

OpenAI `text-embedding-3-small`, 1536 dims, via plain `fetch` (no new npm dependency — matches the self-hosted-fonts / minimal-deps ethos). Chosen because: cheapest at scale (**$0.02 / 1M tokens**), canonical pgvector pairing, server-side only (Vercel → not behind Chris's corporate proxy, so the Google-Fonts-class block doesn't apply). Env var `OPENAI_API_KEY`; model overridable via `PIP_EMBED_MODEL`. Anthropic's officially recommended embeddings partner is **Voyage AI** (`voyage-3.5-lite`) — a clean future swap (same shape, different endpoint), left behind a single `api/_embed.js#embedTexts()` seam so switching is code-light. One provider in code for v1 to stay simple.

Cost math (this is why it's safe to just turn on):

- **Backfill (one-time):** ~~50 meetings × (~~400-tok notes + ~~200-tok summary) + ~30 updates × ~60 tok ≈ **~~32k tokens ≈ $0.0006**. Even at 10× the corpus, < **$0.01** total.
- **Ongoing sync:** a handful of changed sources/day × a few hundred tokens ≈ **pennies/month**.
- **Recall query embeds:** ~20 tokens per chat turn ≈ **$0.0000004** each — noise.
- Net: effectively free. The fingerprint gate guarantees we never pay to re-embed unchanged content, so repeated daily sweeps don't accumulate cost.

RISK CONTROLS (Sanity-Pass Rule — RLS + embeddings are silent-failure surfaces)

1. **No cross-user / cross-account leak.** The recall RPC carries BOTH an explicit `user_id = auth.uid()` predicate AND table RLS, and is only ever called with the user-JWT client (so `auth.uid()` resolves). A leak would require both layers to fail simultaneously. Account scoping is the default; the global lane is still user-scoped. *Runtime trace:* client POST → `Authorization: Bearer <jwt>` → server builds `userClient` with the global header (not just `getUser` validation) → RPC runs as `authenticated` → `auth.uid()` = the user → RLS + predicate both filter to the user's rows.
2. **Missing key degrades to nothing.** No `OPENAI_API_KEY` → `embedTexts()` returns null → `embed-sync` no-ops and recall is skipped. **Pip is never broken by F6** — it simply has no recall until the key is set. (Chris must add `OPENAI_API_KEY` to Vercel env; documented.)
3. **Embed once.** Per-source `content_fingerprint` (same FNV-1a as F3) prevents re-embedding unchanged content, even across liberal daily triggers — the cost guarantee.
4. **Fail toward silence, never error.** Every F6 path (sync, query-embed, RPC) is wrapped so a failure logs + returns empty rather than throwing into a user-facing request. Recall is strictly additive context; its absence costs nothing but a missing nicety.
5. **Dimension integrity.** `vector(1536)` is fixed to the model's native dims; a provider/dim mismatch fails the insert (caught + skipped), never silently stores a wrong-shaped vector.
6. **Hook Order Rule.** `useEmbeddingSync` is called unconditionally above App.jsx's `authLoading` return and no-ops internally when `userId` is null.
7. **Gate every commit:** `npx vite build && npx vitest run && node scripts/check-guards.js && node scripts/test-api-imports.js`. 330-test baseline stays green; new tests add to it. `api/embed-sync.js` registered in `test-api-imports.js`.

8. **DB additive only** (`create … if not exists` , new table + RPC), applied via Supabase MCP
    - folded into `schema.sql` . No destructive change.

BUILD ORDER (one concern per commit, gated between each)

1. **Schema + RPC** — `vector` extension, `folio_embeddings` table + RLS + hnsw index + `match_folio_embeddings` RPC (MCP migration + `supabase/folio_embeddings.sql` + fold into `schema.sql`). No code behavior yet.
2. **Embed seam + recall render** — `api/_embed.js#embedTexts()` (OpenAI fetch, key-optional); `accountContext.js` `recallSection` + `includeRecall` defaults + tests (render-from-hits + surface gating). Pure, no fetch — safe.
3. **Writer** — `api/embed-sync.js` (fingerprint-gated batched embed + upsert) + register in `test-api-imports.js` + `_pipUsage.js` cost-table entry for the embed model.
4. **Trigger** — `src/hooks/useEmbeddingSync.js` + one call in App.jsx (above authLoading).
5. **Reader** — recall query + attach in `api/pip.js` (gated chat/brief); global-lane render in `pipContext.js` .
6. **Docs** — `docs/data-handling.md` (what's embedded + RLS scope — BEFORE shipping behavior), `docs/product-overview.md` (Pip recall capability), `docs/upgrades.md` entry; regenerate PDFs.

OUT OF SCOPE (named so they're not silently dropped)

- After-summarize instant-embed trigger (daily sweep covers recall's old-content target).
- Embedding `gauge_projects.notes`, contacts, tasks, glossary (lower recall value / structured).
- Recall inside summarize + operator (the section exists in the shared builder and is default-off there; wiring their fetch is a later, Chris-tested step — same caution as F3's "retire state_prose" note).
- Voyage AI provider (env-swappable seam left in place; OpenAI for v1).
- F5 (agent loop) — next session.